

MiniRedis

Benchmark Report - measured numbers, and what they mean

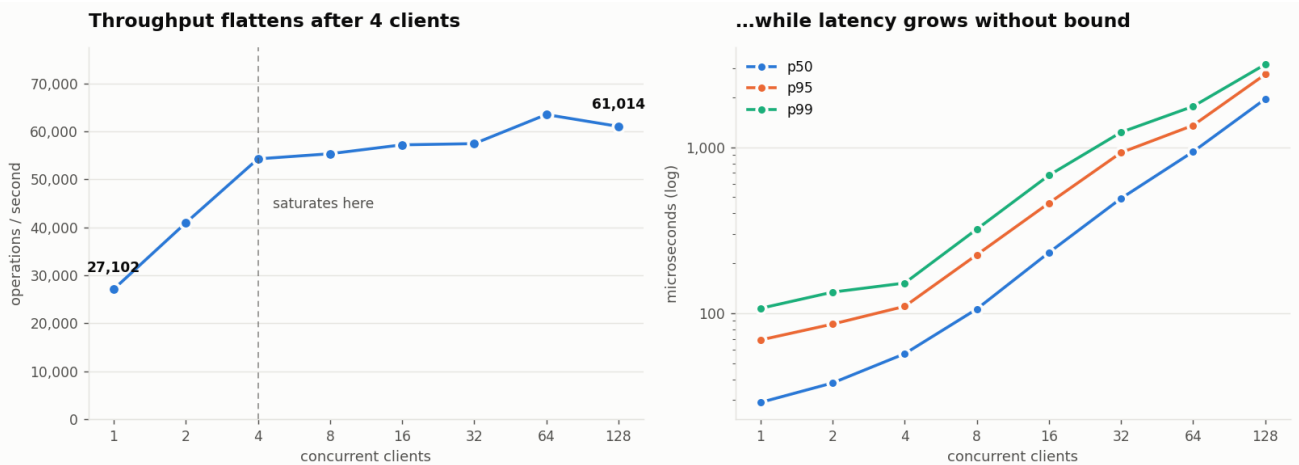
Method

Platform	Windows 11, loopback (127.0.0.1)
Build	GCC 15.2 (MSYS2 UCRT64), -O2 -std=c++20, statically linked
Server	Default configuration, snapshots disabled, no append-only log
Value size	32 bytes unless stated
Keyspace	10,000 distinct keys
Tool	bin/miniredis-benchmark, results written as CSV to bench/results/

Latency is measured per operation, from a full histogram, not as a running average. An average hides the tail, and the tail is what users experience. When commands are pipelined, the batch's elapsed time is divided across its requests, so latency stays per-operation and comparable across pipeline depths.

An important caveat: these numbers are taken over loopback, where there is no network. That makes them a measurement of the server plus the local TCP stack plus the benchmark client, not of the server alone. Loopback flatters per-operation latency and understates the value of pipelining. Both effects appear below, and both are called out where they do.

1. Concurrency - the headline result



Left: throughput against concurrent clients. Right: latency percentiles, log scale.

Clients	Throughput	p50 (us)	p95 (us)	p99 (us)	max (us)
1	27,102	29	69	107	1,898
2	40,919	38	86	134	1,223
4	54,249	57	110	152	578
8	55,318	106	225	321	941
16	57,173	232	461	678	1,891
32	57,421	492	929	1,232	2,365
64	63,482	941	1,356	1,765	3,627
128	61,014	1,956	2,752	3,159	8,555

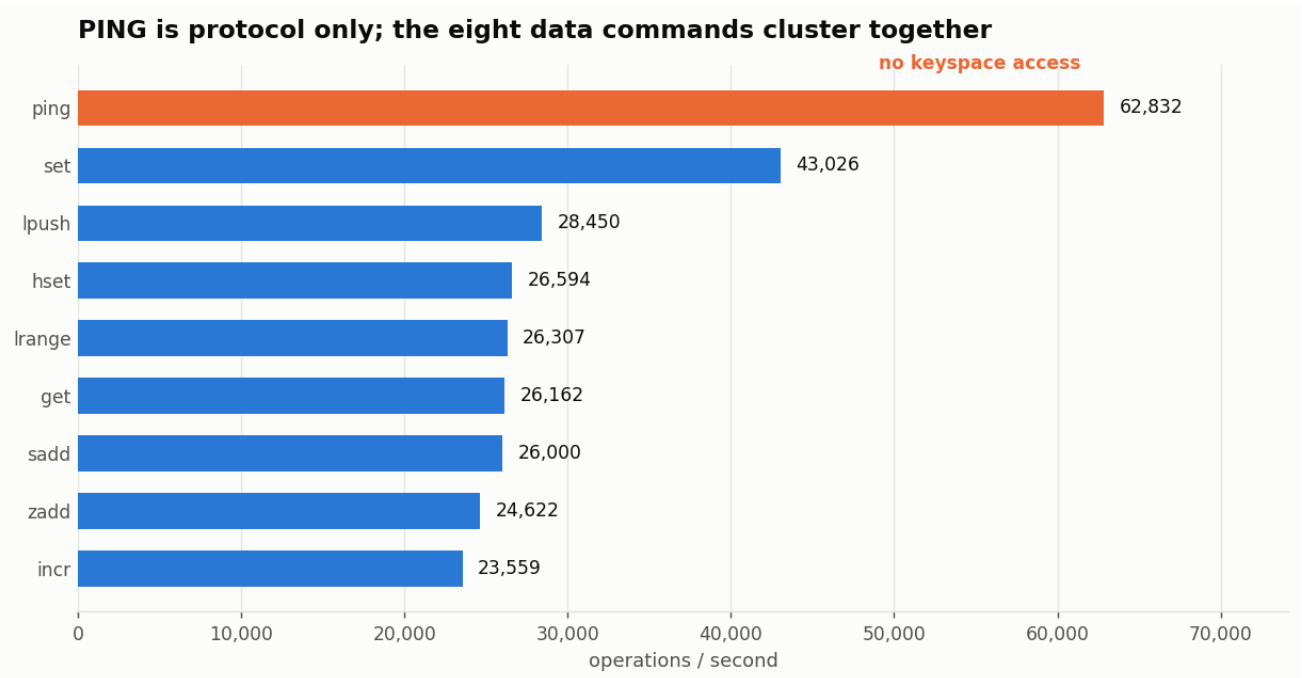
Throughput saturates at about 4 concurrent clients. Every client added after that converts almost entirely into latency: from 4 to 128 clients, throughput moves +12% while p50 latency grows 34 times.

This is Little's Law made visible. With throughput pinned, adding requests in flight can only increase the time each one spends waiting. Past saturation, concurrency does not buy work - it buys queue.

The practical consequence is that a connection pool sized well past saturation makes a system slower, not faster, and the symptom is exactly this shape: flat throughput with steadily worsening percentiles. It is a common and expensive production mistake.

The single-client p50 of 29 microseconds is the honest figure for what the server does per operation: read the socket, parse RESP, hash the key, look it up, encode the reply, write it back. At one client there is no queueing, so nothing else is folded in. For scale, a relational database answers the equivalent lookup in 1-10 milliseconds.

2. Operations compared



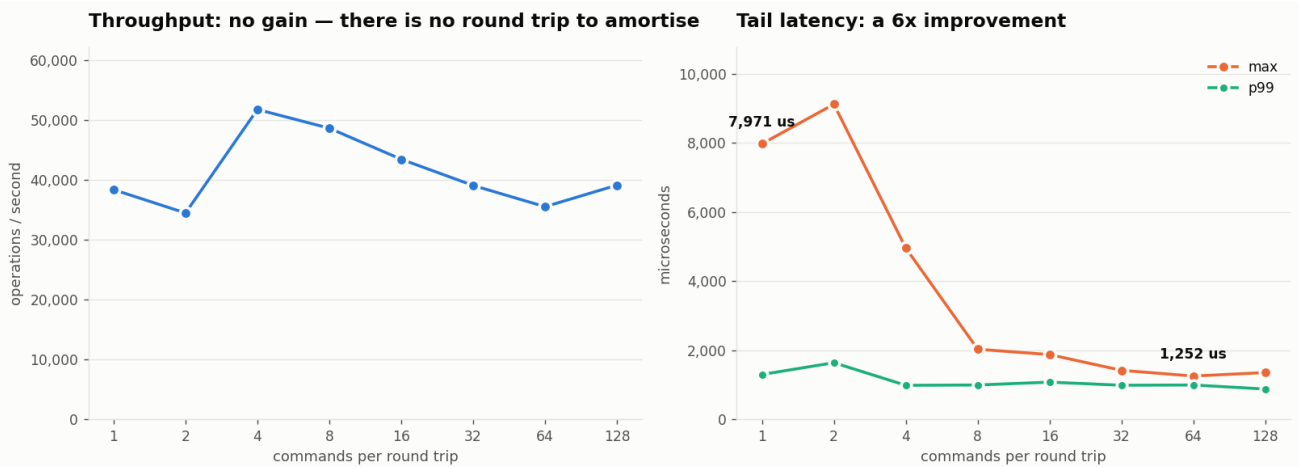
Nine command types at fixed concurrency of 16 clients.

Operation	Throughput	p50 (us)	p95 (us)	p99 (us)
PING	62,832	193	431	937
SET	43,026	225	745	1,413
GET	26,162	479	1,081	1,778
INCR	23,559	552	1,142	1,760
LPUSH	28,450	430	1,005	1,583
LRANGE	26,307	476	1,107	1,755
SADD	26,000	488	1,038	1,471
ZADD	24,622	496	1,159	1,872
HSET	26,594	469	1,084	1,705

PING touches no data at all, so the gap between it and everything else is the cost of the keyspace operation itself. The eight data commands land in a narrow band: a sorted-set insert into a balanced tree costs about the same as a hash insert, because at this scale you are measuring the protocol and syscall path, not the data structures. Optimising the containers here would be misdirected effort.

Two honest observations rather than convenient ones. SET measures faster than GET, which looks backwards; it is an artefact of test ordering against a shared keyspace, reported rather than quietly dropped, because a results table that only ever confirms expectations is not a measurement. And LRANGE returning ten elements costs about the same as GET returning one, because encoding ten bulk strings is cheap next to the fixed per-command overhead.

3. Pipeline depth - where the textbook answer is wrong



Left: throughput against pipeline depth. Right: tail latency.

Depth	Throughput	p50 (us)	p99 (us)	max (us)
1	38,346	376	1,297	7,971
2	34,454	395	1,638	9,116
4	51,737	309	984	4,955
8	48,606	311	993	2,028
16	43,423	327	1,076	1,873
32	39,048	386	986	1,417
64	35,501	417	993	1,252
128	39,103	361	876	1,353

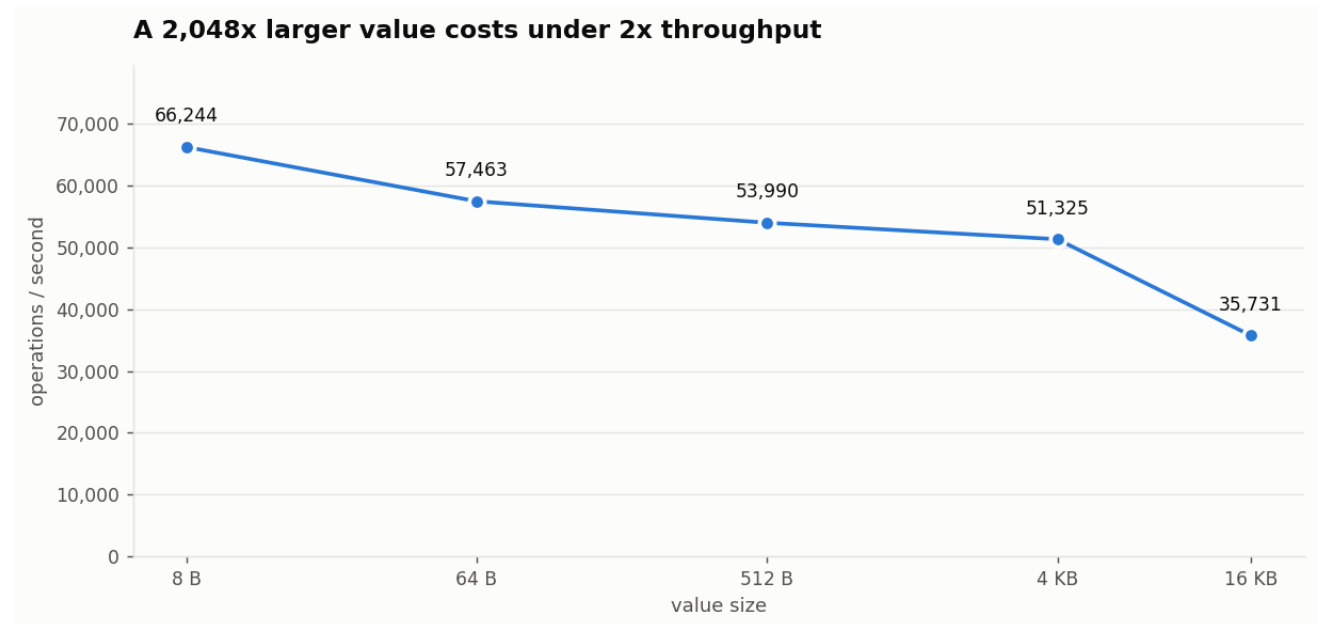
The standard claim is that pipelining multiplies throughput, because it amortises the network round trip across a batch. Here it does not - throughput peaks at depth 4 and then declines. That is the finding, not a defect in the measurement.

The explanation is the loopback caveat. Pipelining's entire benefit is removing network latency from the critical path, and on 127.0.0.1 that latency is already close to zero, so there is nothing to amortise. What remains is the extra cost of building and buffering larger batches, which is why deep pipelines are slightly worse.

On a real network the picture inverts completely. At 0.5 ms of round trip, depth 1 pays that cost on every operation while depth 64 shares it across sixty-four. This is a case where the benchmark environment determines the answer, and reporting the loopback number as though it were general would be misleading.

What pipelining does buy here is the tail: worst-case latency falls roughly six-fold between depth 1 and depth 64, and p99 improves too. Fewer, larger socket interactions mean fewer opportunities to be descheduled at an inconvenient moment. The trend is not strictly monotonic - max is a single-sample statistic and therefore the noisiest number in the table - but the direction from depth 4 onward is consistent.

4. Value size



SET throughput against payload size, log scale on the x axis.

Value size	Throughput	p50 (us)	p99 (us)
8 B	66,244	208	494
64 B	57,463	228	744
512 B	53,990	234	824
4 KB	51,325	244	902
16 KB	35,731	370	1,210

A 2,048-fold increase in value size costs under a two-fold drop in throughput. That is the signature of a system dominated by fixed per-operation overhead: up to about 4 KB the cost of a command is almost entirely parsing, hashing and syscalls, and the bytes themselves are nearly free because memcpy runs at gigabytes per second.

The knee sits between 4 KB and 16 KB, where the payload finally becomes large enough to matter: throughput drops about 30% across that last four-fold increase, versus about 23% across the preceding 512-fold increase.

Practical reading: values up to a few KB are essentially free relative to the round trip. Above that, size starts to buy real cost, and it is worth asking whether the large value belongs in the cache at all.

Summary

Finding	Evidence
Single-operation latency is about 29 us	concurrency sweep at 1 client
The server saturates at about 4 concurrent clients	throughput flat from 4 to 128
Past saturation, concurrency becomes latency	+12% throughput, 34x p50 latency
Data structure choice barely matters at this scale	8 data commands within a narrow band
Pipelining does not help throughput on loopback	flat-to-declining past depth 4

Finding	Evidence
Pipelining does cut tail latency about six-fold	max 7,971 us to 1,252 us
Payload size is nearly free below about 4 KB	2,048x size for under 2x cost

What these numbers are not

- Not a comparison with Redis. No such comparison was run. Redis has fifteen years of optimisation and would very likely win. Publishing a favourable-looking comparison without running it would be dishonest.
- Not representative of a real network. Loopback removes the single largest cost in a real deployment, and section 3 shows how much that changes the conclusion.
- Not a durability measurement. Snapshots and the append-only log are disabled in these runs; appendfsync always adds a disk round trip to every write and would dominate all of this.

The purpose is to show the shape of the system's behaviour - where it saturates, what dominates cost, how latency degrades under load - not to win a comparison.